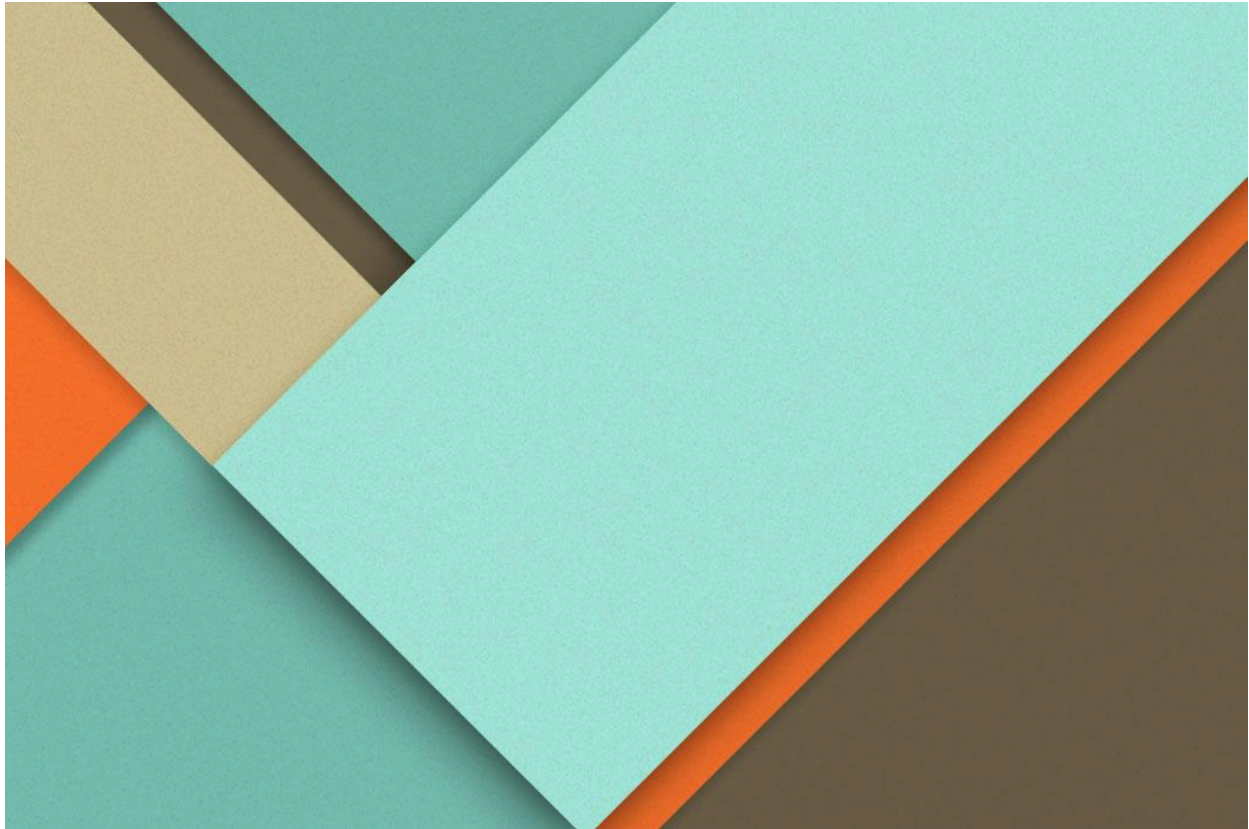




Motores de Juego 2D y 3D

13/04/2026

—

José Manuel Sánchez Rosal

2º DAM

IES Antonio Gala

1400 Palma del Río (Córdoba)

Índice

Índice.....	1
1. Idea del juego y descripción general.....	1
2. Lógica del juego (reglas y funcionamiento).....	2
3. Explicación de los elementos del juego.....	3
4. Capturas del desarrollo y ejecución.....	4
4.1 Árbol de nodos.....	4
4.2. Juego en ejecución.....	5
4.3. Código Fuente.....	6
4.4. Game Over.....	7
5. Problemas encontrados y soluciones aplicadas.....	8
5.1. Secuestro de eventos de teclado (Focus Stealing) en la Interfaz UI.....	8
5.2. Colisiones y desbordamiento de memoria ("Index Out of Bounds") durante las Rotaciones.....	8
5.3. Interrupciones auditivas por concurrencia en el flujo de sonido.....	9
6. Reflexión final sobre el proceso.....	9

1. Idea del juego y descripción general

"**Tetroll**" es un videojuego de lógica y reflejos inspirado en la mecánica clásica de "**Tetris**". El proyecto nace con un doble objetivo: aplicar en la práctica conceptos avanzados de programación (matrices 2D, bucles de juego) y ofrecer un producto de software final pulido y moderno.

Su principal característica técnica es el diseño **Mobile-First**. Desarrollado con el motor Godot 4, el juego está construido íntegramente en torno a la interacción táctil en dispositivos móviles, utilizando resolución vertical y botones adaptables. En conclusión, más que un simple clon, es una adaptación pedagógica que demuestra la viabilidad de crear videojuegos móviles 2D empleando herramientas de código abierto.

2. Lógica del juego (reglas y funcionamiento)

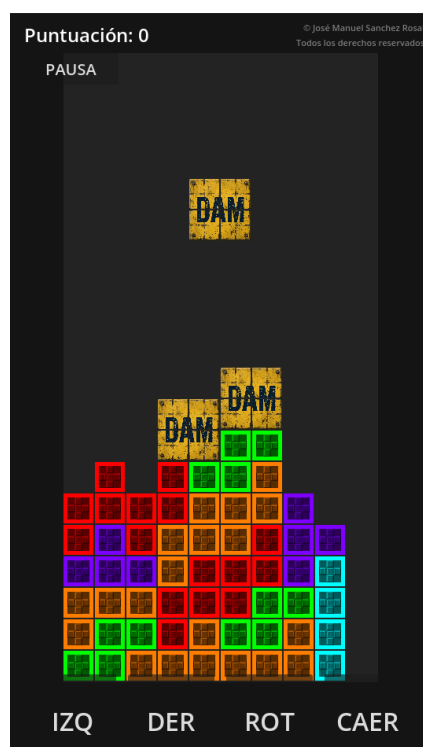
La jugabilidad de Tetroll se apoya en un sistema de reglas estrictas, ejecutadas en tiempo real y evaluadas en cada fotograma del motor (a través de la función `_process` y temporizadores de ciclo). El funcionamiento interno se describe de la siguiente manera:

- **El Tablero Virtual y la Gravedad:** El espacio de juego no utiliza físicas reales de gravedad, sino un sistema discreto basado en una cuadrícula virtual de **10 columnas por 20 filas (10x20)**. Un temporizador interno (FallTimer) dicta el ritmo de "caída". Cuando el temporizador llega a cero, el bloque activo desciende exactamente una celda hacia abajo (desplazamiento en el eje Y positivo).
- **Generación de Piezas (Spawning):** El juego escoge de forma pseudoaleatoria uno de los 7 Tetrominós estándar (I, O, T, S, Z, J, L). Esta pieza se "instancia" lógicamente en la parte superior central de la cuadrícula. Cada bloque está conformado estrictamente por 4 sub-cuadrados.
- **Interacción y Movimiento:** El jugador tiene la capacidad de intervenir en la caída de tres formas. Puede **desplazar** la pieza lateralmente (eje X) siempre y cuando la celda de destino esté libre y dentro de los límites. Puede **rotar** la pieza 90 grados matemáticamente aplicando una matriz de transformación simple sobre las coordenadas de sus 4 bloques. Finalmente, puede activar una "**caída rápida**" (Hard Drop) que altera el temporizador para precipitar la pieza hasta el suelo de forma casi instantánea.
- **Limpieza de Líneas (Line Clearing) y Puntuación:** Una vez que una pieza toca el suelo o un bloque inferior y no puede seguir cayendo, se "bloquea" (lock_piece). En ese instante, el algoritmo escanea todas las filas desde abajo hacia arriba. Si detecta que alguna fila de 10 columnas está completamente llena de bloques y no contiene ningún valor nulo, la línea se elimina. Los bloques situados por encima de la línea eliminada se desplazan exactamente una fila hacia abajo. Por cada línea eliminada, se otorgan puntos al jugador (p. ej., 100 puntos por línea) utilizando un *Singleton* (Autoload).
- **Curva de Dificultad y Condición de Derrota:** A medida que la puntuación global supera ciertos umbrales predefinidos, la variable de tiempo de espera del temporizador de caída se reduce, haciendo que el juego sea progresivamente más rápido y difícil. El juego concluye inevitablemente en "**Game Over**" cuando las piezas se han apilado de tal forma que el punto de aparición (spawn) en la fila 0 **colisiona con un bloque previamente fijado**, impidiendo que entre una nueva pieza al tablero.

3. Explicación de los elementos del juego

Para construir la experiencia sin depender de assets externos pesados que comprometan el rendimiento móvil, el proyecto se ha estructurado utilizando exclusivamente los nodos nativos y modulares de Godot:

- **Integración de Sprites Propios y Personalización:** Para dotar al videojuego de un acabado visual superior y cumplir con los requisitos de diseño ("**Sprite propio correcto y original**"), se ha sustituido el renderizado primario de figuras geométricas lisas por un sistema de texturas dinámicas mediante nodos TextureRect y AtlasTexture.
- **Paleta de Colores y Efecto 3D:** Se ha incorporado un recurso gráfico propio en escala de grises (**block_sprite.jpg**) con un suave biselado interior. El motor de Godot tiñe dinámicamente este único asset (mediante la propiedad modulate) para generar todas las piezas con un efecto tridimensional (3D). Cada forma tiene un color RGB específico altamente contrastante (Cyan para la pieza larga I, Morado para la T, etc.), lo que mejora significativamente el feedback visual táctil para el jugador en pantallas móviles.
- **Mapeo de Texturas Atlas (Logo "DAM"):** Como elemento distintivo de originalidad, se ha programado una lógica especial para la pieza Cuadrada (pieza 'O'). Se ha diseñado una textura única (**dam_sprite.jpg**) que contiene las siglas del ciclo "DAM". Mediante cálculos matemáticos matriciales, el código "trocea" esta textura en cuatro cuadrantes durante el tiempo de ejecución. Cada sub-bloque de la pieza carga de forma independiente su fracción correspondiente de la imagen. Así, al unirse, proyectan el logo completo en pantalla y, en caso de eliminarse una línea inferior del tablero, el motor físico permite que la mitad superior del logo caiga por gravedad sin corromper el renderizado, respetando la mecánica intrínseca del Tetris clásico de manera impecable:



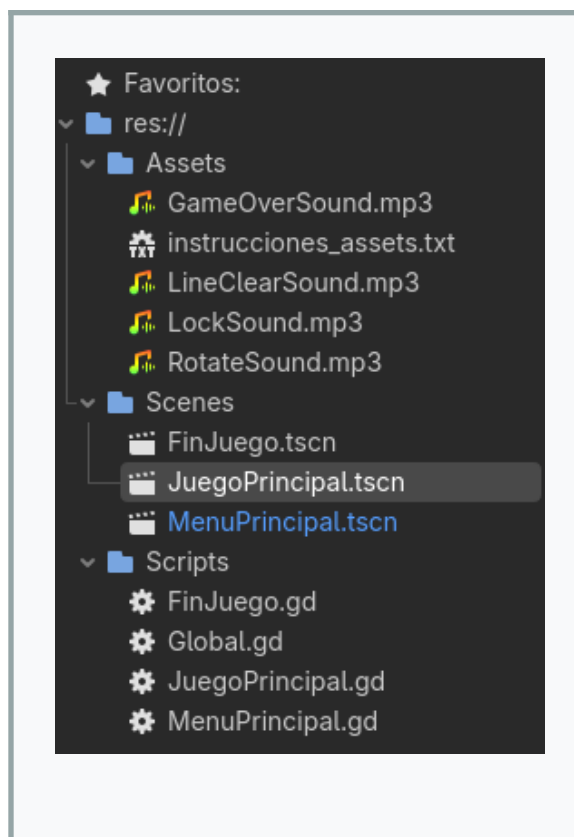
- **Interfaz de Usuario Independiente (CanvasLayer y Control):** Para evitar que los elementos de puntuación y los botones se vean afectados por el escalado de resolución, se encapsulan en un **CanvasLayer**. En la parte inferior, se aloja un **HBoxContainer** que contiene los nodos Button ("IZQ", "DER", "ROT", "CAER"). Este contenedor es responsivo: espacia de manera automática y equidistante los botones para ocupar todo el ancho disponible de la pantalla, garantizando un área táctil lo más amplia posible.
- **Gestor de Escenas y Persistencia de Datos (Global.gd):** Se utiliza el patrón **Singleton (Autoload en Godot)** mediante un script global. Este elemento invisible funciona como el administrador del juego: guarda la puntuación obtenida en memoria para que no se pierda al cambiar de escena, y contiene las funciones para saltar entre la escena de "Menú Principal", el "Juego", y la pantalla final de "Game Over".
- **Sistema de Audio Basado en Eventos (AudioStreamPlayer):** Los efectos de sonido no son meramente decorativos, sino que proveen retroalimentación sensorial (Feedback). Múltiples reproductores de audio independientes están asignados a eventos concretos: rotación de la pieza, encaje exitoso en el suelo (lock) y limpieza de línea. Se han cargado de forma dinámica mediante la función load() para asegurar que la memoria de audio se gestione eficientemente.

4. Capturas del desarrollo y ejecución

A continuación se documentan las distintas facetas del desarrollo técnico y estético del producto final.

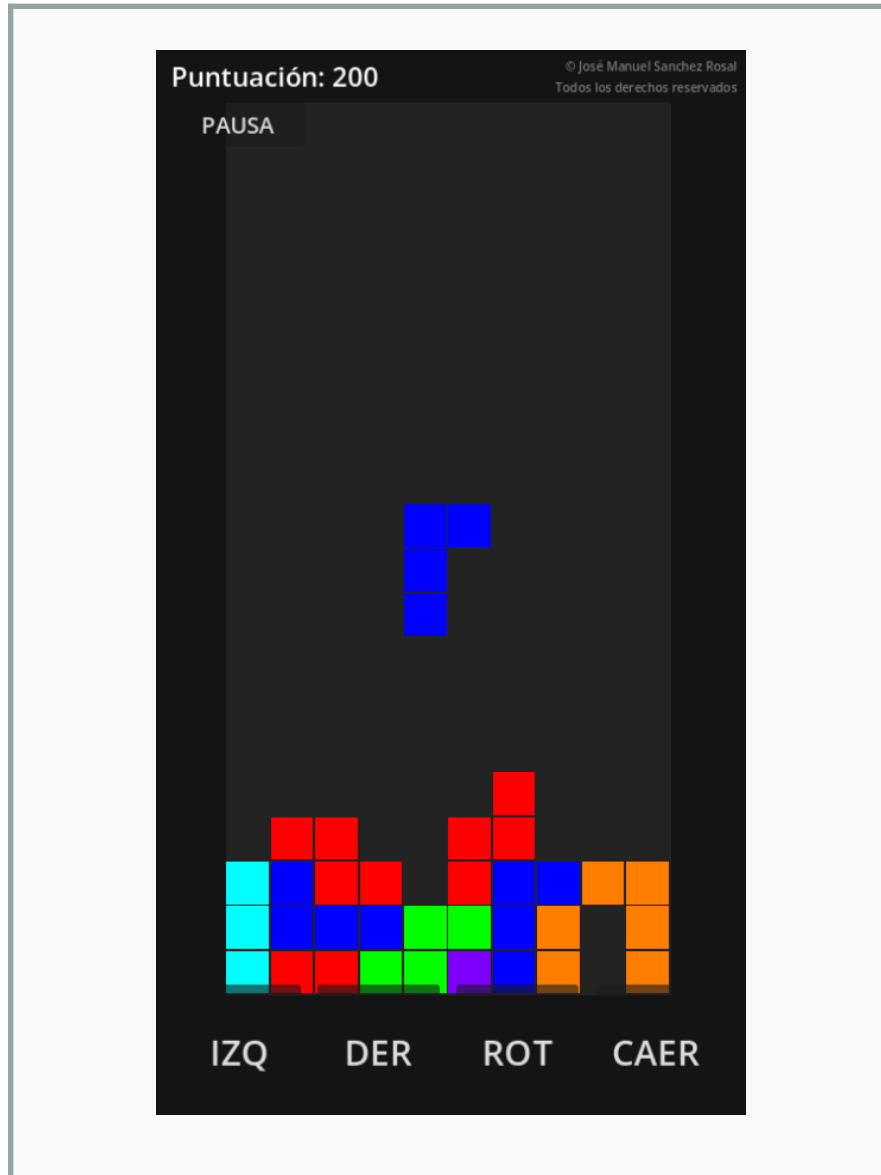
4.1 Árbol de nodos

En esta captura se detalla la estructura y organización interna del proyecto dentro del motor Godot 4. Se aprecia una clara separación modular en directorios: la carpeta **Assets** contiene los recursos de audio importados dinámicamente; la carpeta **Scenes** almacena las distintas pantallas del juego (Menú, Juego Principal y Fin de Juego); y la carpeta **Scripts** organiza los archivos de código en GDScript, evidenciando un desarrollo ordenado y escalable:



4.2. Juego en ejecución

Captura de una partida en curso demostrando el correcto funcionamiento del bucle de juego. Se aprecian las piezas (Tetrominós) formadas por bloques de colores encajadas en la parte inferior de la cuadrícula virtual. En la zona superior se muestra la puntuación actual gestionada por el **Singleton**, y en la parte inferior destaca la **botonera táctil responsiva** (IZQ, DER, ROT, CAER), evidenciando la aplicación directa del paradigma **Mobile-First** orientado a dispositivos móviles:



NOTA: Se adjunta videodemo de una partida.

4.3. Código Fuente

Fragmento del script principal ([JuegoPrincipal.gd](#)) escrito en GDScript. En esta sección del código se aprecia la declaración de las constantes fundamentales que definen las dimensiones de la cuadrícula (10 columnas por 20 filas). Asimismo, se muestran las matrices vectoriales que conforman matemáticamente la estructura de cada una de las 7 piezas clásicas del juego, junto con la asignación de sus respectivos colores representativos mediante el objeto **Color**:

```
extends Node2D

const COLS = 10
const ROWS = 20
const CELL_SIZE = 40

var grid = [] # Matriz 2D de ColorRects (null si está vacío)

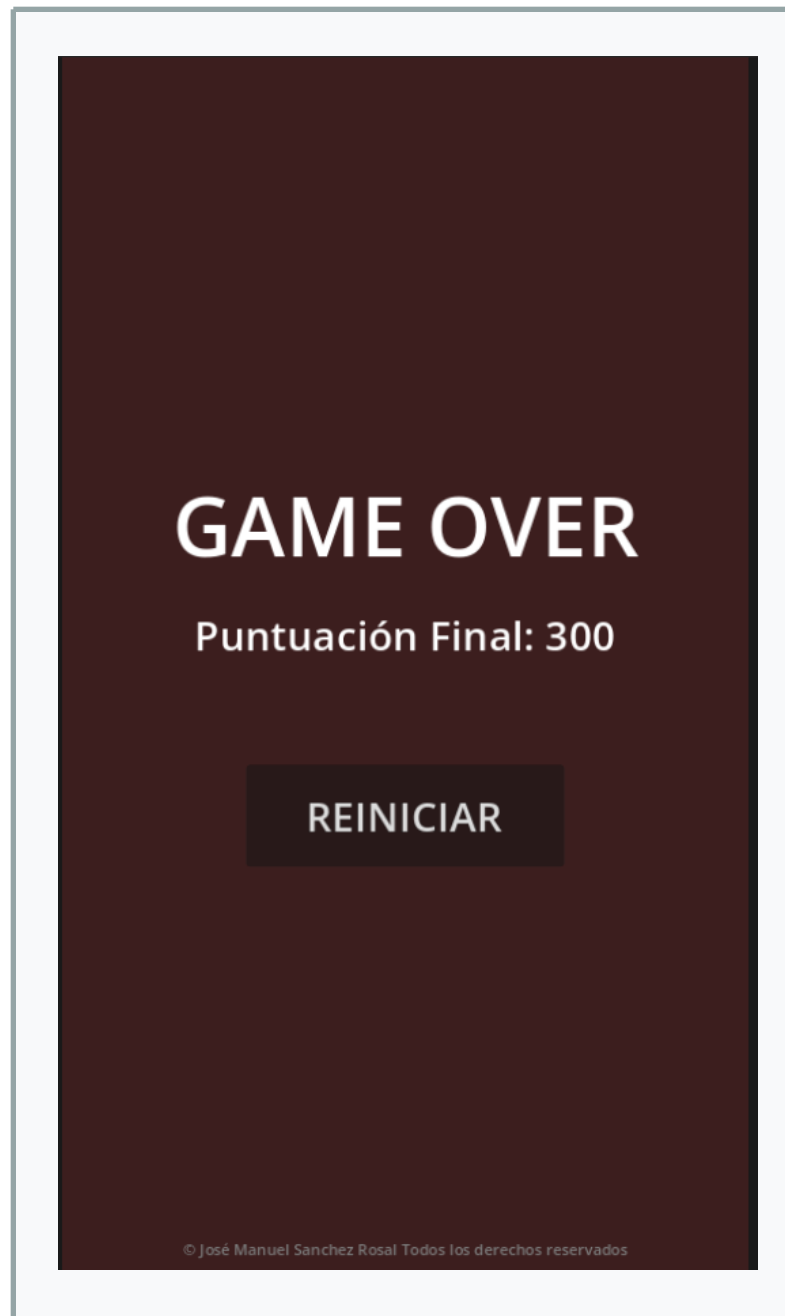
# Formas de los Tetrominós
const SHAPES = [
    [Vector2(-1, 0), Vector2(0, 0), Vector2(1, 0), Vector2(2, 0)], # I
    [Vector2(-1, -1), Vector2(-1, 0), Vector2(0, 0), Vector2(1, 0)], # J
    [Vector2(1, -1), Vector2(-1, 0), Vector2(0, 0), Vector2(1, 0)], # L
    [Vector2(0, -1), Vector2(1, -1), Vector2(0, 0), Vector2(1, 0)], # O
    [Vector2(0, -1), Vector2(1, -1), Vector2(-1, 0), Vector2(0, 0)], # S
    [Vector2(0, -1), Vector2(-1, 0), Vector2(0, 0), Vector2(1, 0)], # T
    [Vector2(-1, -1), Vector2(0, -1), Vector2(0, 0), Vector2(1, 0)] # Z
]

# Colores para cada Tetrominó
const COLORS = [
    Color(0.0, 1.0, 1.0), # I (Cyan)
    Color(0.0, 0.0, 1.0), # J (Azul)
    Color(1.0, 0.5, 0.0), # L (Naranja)
    Color(1.0, 1.0, 0.0), # O (Amarillo)
    Color(0.0, 1.0, 0.0), # S (Verde)
    Color(0.5, 0.0, 1.0), # T (Morado)
    Color(1.0, 0.0, 0.0) # Z (Rojo)
]

var active_piece_type: int
var active_piece_cells: Array
var active_pos: Vector2
var fast_drop: bool = false
var normal_wait_time: float = 0.5
```


4.4. Game Over

Pantalla final del juego ([FinJuego.tscn](#)) a la que se accede una vez que el jugador alcanza la condición de derrota preestablecida (las piezas se apilan hasta colisionar con el límite superior de la pantalla). En esta interfaz se muestra de forma clara la puntuación final obtenida por el jugador, valor que ha sido persistido y recuperado a través del script global (Autoload). También se incluye un botón interactivo para reiniciar el ciclo de juego de forma inmediata:



5. Problemas encontrados y soluciones aplicadas

Durante las fases de integración y prototipado surgieron múltiples **retos técnicos**. A continuación se exponen tres de los problemas más críticos y el enfoque utilizado para su resolución:

5.1. Secuestro de eventos de teclado (Focus Stealing) en la Interfaz UI

Problema: Al implementar soporte híbrido (para permitir controles de PC mediante teclado y controles móviles a través de botones táctiles en pantalla), se descubrió que al pulsar las flechas direccionales físicas, el juego dejaba de mover la pieza activa. En su lugar, el sistema nativo de Godot interceptaba las flechas para "navegar" visualmente entre los botones "IZQ", "DER", etc., dibujando un contorno azul a su alrededor. El teclado físico controlaba la UI, pero no el juego.

Solución: Se realizó un ajuste profundo en las propiedades de la jerarquía de los nodos Button. Se modificó la propiedad **focus_mode** de todos los botones implicados, cambiándola del valor predeterminado (*Focus All*) al valor **None (0)**. Al despojar a los botones de su capacidad para recibir el foco del teclado, los eventos de *Input* físicos volvieron a propagarse correctamente hacia la función `_process` del script principal, permitiendo el uso simultáneo e ininterrumpido de teclado táctil y teclado físico.

5.2. Colisiones y desbordamiento de memoria ("Index Out of Bounds") durante las Rotaciones

Problema: El sistema de movimiento lateral contaba con barreras de seguridad simples, pero la función de *rotación* presentaba fallos graves al estar cerca de las paredes. Al presionar "Rotar" cuando una pieza larga (como la "I") estaba rozando la pared derecha, la transformación matemática de 90 grados movía las coordenadas relativas de sus bloques fuera del rango válido de la matriz (ej. intentando acceder a la celda $x = 11$ en un tablero de ancho 10). Esto corrompía la memoria y forzaba un cierre repentino de la aplicación.

Solución: Se rediseñó la función de rotación para aplicar un **modelo predictivo y especulativo**. Antes de sobrescribir la matriz real de la pieza, el sistema almacena las coordenadas deseadas en un array temporal (`new_cells`). Luego, llama a la función **check_collision()** y le pasa esas coordenadas futuras simuladas. Solo si la función de colisión confirma que *todas* las celdas nuevas caen dentro de los límites estrictos ($x \geq 0$, $x < COLS$, $y < ROWS$) y que no chocan con bloques bloqueados, la rotación es aprobada y aplicada visualmente. Si se detecta un error, la rotación simplemente es abortada en silencio sin causar fallos.

5.3. Interrupciones auditivas por concurrencia en el flujo de sonido

Problema: En las primeras versiones del prototipo, se implementó un único nodo **AudioStreamPlayer** para gestionar todo el sonido, cambiando su propiedad stream sobre la marcha. Sin embargo, al jugar a velocidades altas, si un jugador rotaba la pieza e inmediatamente forzaba una caída rápida encajándola contra el suelo, el sonido de "rotación" se cortaba abruptamente a la mitad para dar paso al sonido de "encaje", creando una experiencia sonora fragmentada y saturada.

Solución: Se abandonó el modelo de reproductor único en favor de una **arquitectura de canales de audio paralelos**. Se añadieron al árbol de nodos múltiples **AudioStreamPlayer** independientes, cada uno dedicado permanentemente a un único recurso (RotateSound, LockSound, LineClearSound). De este modo, al producirse dos acciones consecutivas de forma casi simultánea, Godot utiliza el motor de mezcla (mixer) interno para solapar ambos archivos de audio en canales distintos, preservando las estelas de sonido (reverb/decay) y otorgando un acabado profesional al proyecto.

6. Reflexión final sobre el proceso

El desarrollo integral de "Tetroll" ha sido un proceso formativo altamente satisfactorio que ha permitido aplicar soluciones lógicas puras a problemas de diseño espacial. Este proceso ha permitido extraer conclusiones clave sobre la arquitectura de software y la planificación **orientada a dispositivos móviles** que se detallan a continuación:

- **Desacoplamiento lógico-visual:** La lección más valiosa fue separar la matemática de la representación gráfica. Entender la pantalla como un simple "lienzo" que lee un array bidimensional facilitó mucho la corrección de errores; si el GDScript es correcto, el apartado visual funciona solo.
- **La potencia de Godot 4:** Su enfoque de jerarquía de nodos permitió separar la cuadrícula del juego y la interfaz (CanvasLayer) para que no interfirieran entre sí. Además, el uso de Señales (Signals) para conectar la UI con las mecánicas garantizó un código limpio, desacoplado y escalable.
- **Mentalidad Mobile-First:** El desarrollo demostró que la adaptación a dispositivos móviles debe planificarse desde el primer día. Preparar un sistema de controles híbridos evitó tener que reescribir la gestión de entradas al final.